COMPUTER VISION

RECOGNITION WITH GLOBAL FEATURES

Ngo Quoc Viet-2025

CONTENTS

LECTURE OUTCOMES

- Understanding color histogram (including multi-dimension histogram), color derivatives and their applications.
- Implementing histogram backprojection to focus the specific object in an image or video.
- Using color histogram to compare the images and implementing the simplest classifier for images.
- Implementing the lecture project to detect brands in image/video.

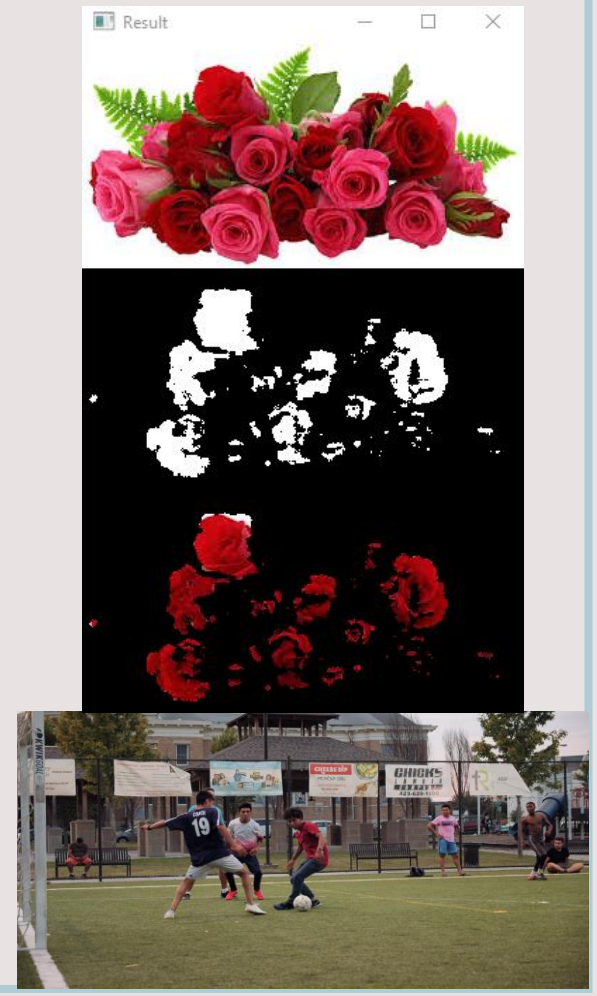


IMAGE FORMATION: BASICS

- Image f(x,y) is characterized by 2 components
  - Illuminationi(x,y)= Amount of source illumination incident on scene
  - Reflectancer(x,y)= Amount of illumination reflected by objects in the scene

$$\boldsymbol{f(x,y) = i(x,y)r(x,y)}$$

where $0 < i(x,y) < \infty$ and $0 \leq r(x,y) \leq 1$

r(x,y) depends on object properties. r = 0 means total absorption and 1 means total reflectance.

- For slowly varying illumination, i(x,y) will be the same at neighboring locations
- Ratio of neighboring locations is independent of illumination

$$\frac{f(x,y)}{f(x+1,y)} = \frac{i(x,y)r(x,y)}{i(x,y)r(x+1,y)} = \frac{r(x,y)}{r(x+1,y)}$$

- Taking logarithms: $f(x,y) - \ln f(x+1,y) = \ln \mathrm{r}(x,y) - \ln \mathrm{r}(x+1,y)$. This is just the derivative of the logarithm of image.

APPEARANCE BASED RECOGNITION

- Appearance-based image recognition is a branch of computer vision that focuses on identifying objects or patterns in images based on their **visual appearance** (visual features of an object rather than its structural or semantic characteristics).
- Appearance-based methods are commonly used in various applications, including object recognition, image classification, and facial recognition.
- Some concepts and techniques associated with appearance-based image recognition: *Histogram*, *Feature Extraction* (HOG, SIFT); *Image Descriptor Representations* of the extracted features that can be used for matching and recognition; *Image Matching* to compare and find similarities between different images. Matching methods can include template matching, nearest-neighbor search, and machine learning-based approaches.

GLOBAL FEATURES

- Global features capture information about the overall structure and content of an entire image, as opposed to local features that focus on specific regions or keypoints
- **Color Histograms**: Represents the distribution of colors in an image.
- **Texture Features**: Describes the overall texture pattern in an image such as Haralick texture features, Gabor filters, and Local Binary Patterns (LBP).
- **Shape Descriptors**: Represents the overall shape of objects in an image such as Hu Moments, Zernike Moments, and Fourier Descriptors.
- **Global Binary Patterns**: Represents the spatial arrangement of pixel values in a binary form such as LBP computed globally for the entire image.
- **Edge Histograms**: Represents the distribution of edge orientations in an image. Captures information about the overall structure of edges.

IMAGE HISTOGRAM

- A histogram is a graph that shows frequency of anything.
- Histogram of an image, like other histograms also shows frequency. But an image histogram, shows frequency of pixels intensity values.
- Histograms has many uses in image processing
  - Analysis of the image (draft prediction, Its like looking an x ray of a bone of a body).
  - Enhancement purposes (contrast, brightness).
  - Thresholding (in computer vision).

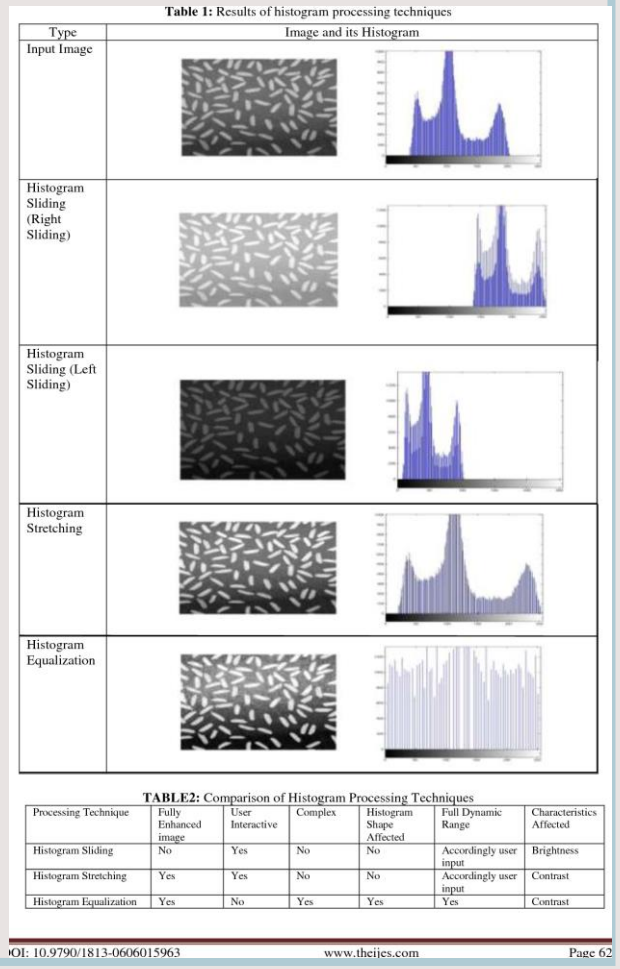


Table 1: Results of histogram processing techniques

Type	Image and its Histogram
Input Image	
Histogram Sliding (Right Sliding)	
Histogram Sliding (Left Sliding)	
Histogram Stretching	
Histogram Equalization	

TABLE2: Comparison of Histogram Processing Techniques

Processing Technique	Fully Enhanced image	User Interactive	Complex	Histogram Shape Affected	Full Dynamic Range	Characteristics Affected
Histogram Sliding	No	Yes	No	No	Accordingly user input	Brightness
Histogram Stretching	Yes	Yes	No	No	Accordingly user input	Contrast
Histogram Equalization	Yes	No	Yes	Yes	Yes	Contrast

DOI: 10.9790/1813-0606015963 www.theijes.com Page 62

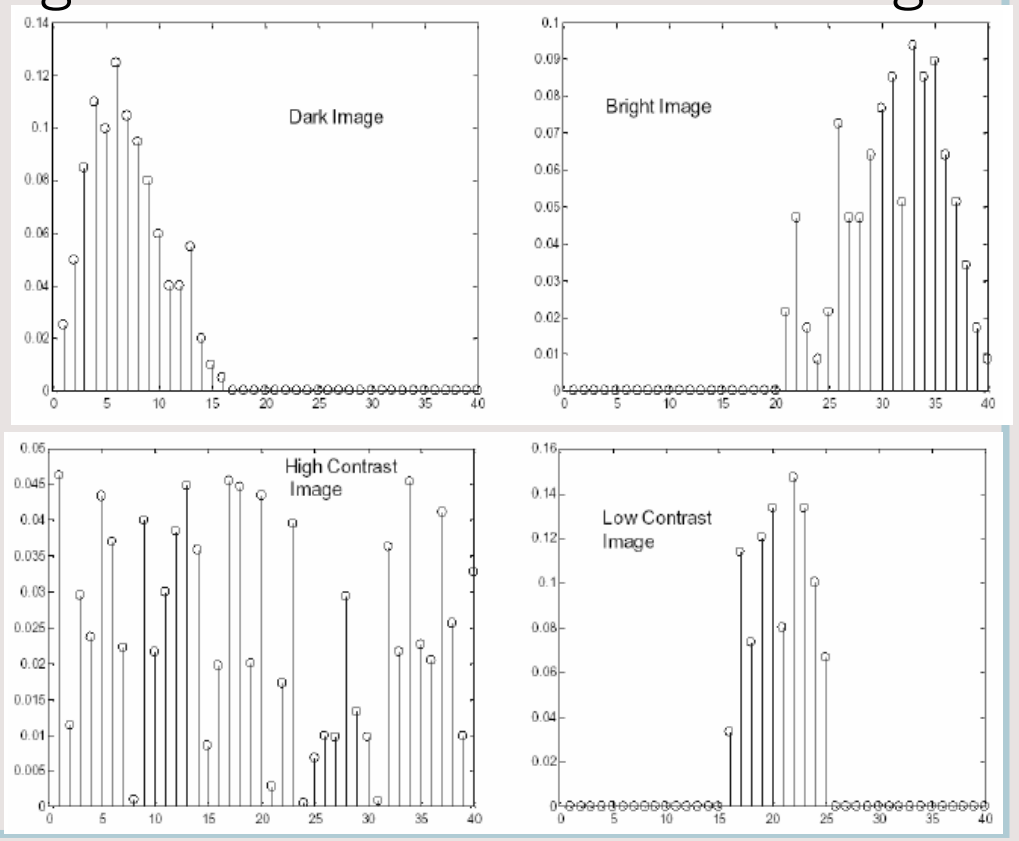
IMAGE HISTOGRAM

- The gray level frequency g of image I is the number of pixels with value g. Histogram is a chart of gray levels in an image. For example, for image I, the histogram h(g) of I is

$$I = \begin{pmatrix} 1 & 2 & 0 & 4 \\ 1 & 0 & 0 & 7 \\ 2 & 2 & 1 & 0 \\ 4 & 1 & 2 & 1 \\ 2 & 0 & 1 & 1 \end{pmatrix}$$

g	0	1	2	4	7
h(g)	5	7	5	2	1

The histogram shape represents the brightness and contrast of the image

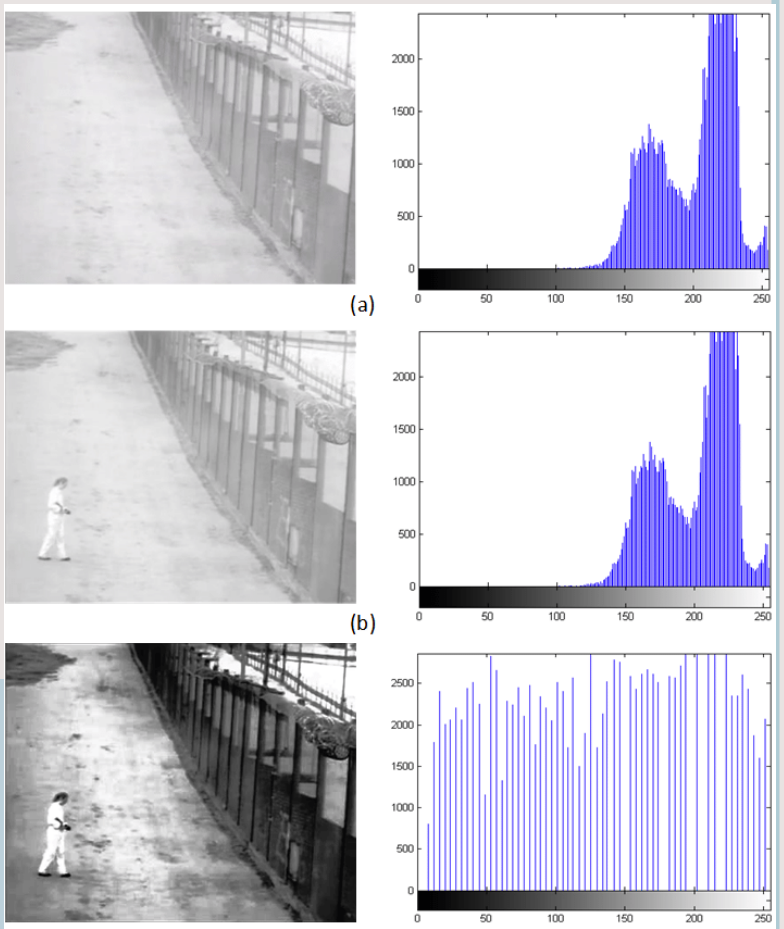


HISTOGRAM EQUALIZATION

- Why histogram equalization?
  - Low contrast image: narrow luminance range.
  - Under-exposed image: concentrating on the dark side.
  - Over-exposed image: concentrating on the bright side.

  → **Unbalanced** histogram do not fully utilize the dynamic range
- **Balanced** histogram gives more pleasant look and reveals rich details.

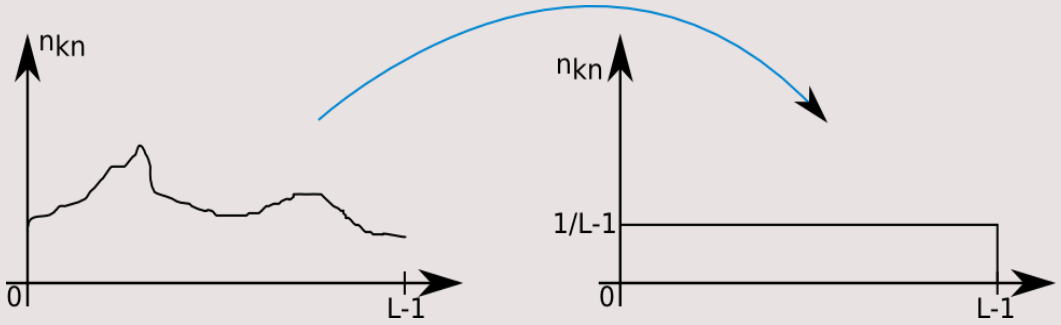
```
# Histograms Equalization of a image using cv2.equalizeHist()
cv.calcHist(img, [0, 1],  None,  [0, 256],  [0, 256] )
equ = cv2.equalizeHist(img)
```



Source: Researchgate

OBJECTIVES OF HISTOGRAM EQUALIZATION

- Change the gray intensity of each pixel to get a new image with balanced histogram.
- Map the each luminance level to a new value such that the output image has approximately **uniform distribution** of gray levels.



$$n_{kn} = \frac{n_k}{length \times width} = p_r(r_k)$$

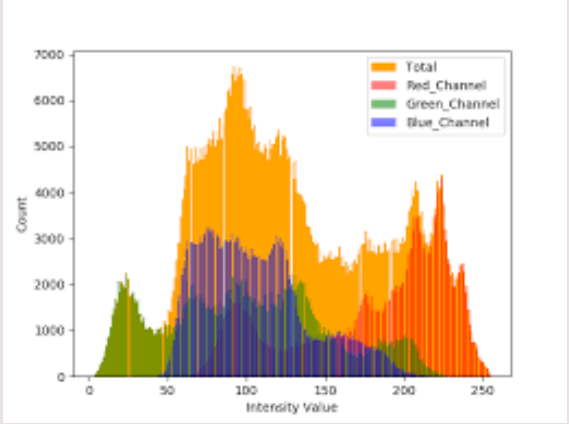
- Two requirements
  - Monotonic (non-decreasing) function: no value reversals.
  - The output range being the same as the input range.

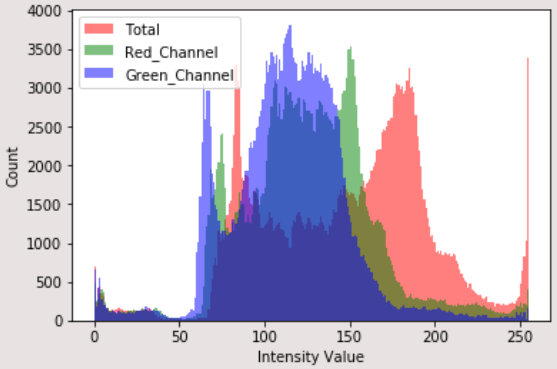
$$\begin{cases} [0, L-1] \to N \\ x \to Card(x) \end{cases} \Rightarrow \begin{cases} [0, L-1] \to [0, 1] \\ x \to pdf(x) = \dfrac{Card(x)}{\sum_{i=0}^{L-1} Card(x_i)} \end{cases}$$

- How? Use **cumulative probability distribution (CDF)** function to equalize.
- Histogram equalization is very useful for scientific images like thermal, satellite or x-ray images.

MULTIDIMENSION HISTOGRAM

- Multidimensional histogram is a representation that captures the distribution of pixel values in an image across multiple dimensions (multiple features or channels simultaneously For example, in an RGB color space, you might create a 3D histogram where the three axes correspond to the intensity values in the red, green, and blue channels.
- Multidimensional histograms are the extension of the concept of Image Histogram to Multi Channel images
- Multidimensional histograms are often used as image descriptors in computer vision tasks. For example, Histogram of Oriented Gradients (HOG) can be considered a type of multidimensional histogram (Lecture 5). But Haar-like, SIFT are not represented as a traditional multidimensional histogram.





MULTIDIMENSION HISTOGRAM

- Multidimensional histograms are often used as image descriptors in computer vision tasks (HOG, Lecture 5).
- Multidimensional histograms can be used **to compare the similarity between images based on their color or texture content**.

```
def calculate2DHistogram(image):
    # Convert the image to HSV color space
    hsv_image = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
    # Calculate 2D histogram for the H and S channels
    hist = cv2.calcHist([hsv_image], [0, 1], None, [180, 256], [0, 180, 0, 256])
    # Normalize the histogram
    hist = cv2.normalize(hist, hist, 0, 1, cv2.NORM_MINMAX)
    return hist
```

```
def compareHistograms(hist1, hist2, method=cv2.HISTCMP_CORREL):
    # Compare histograms using a specified method (default is correlation)
    similarity = cv2.compareHist(hist1, hist2, method)
    return similarity
```

MULTIDIMENSION HISTOGRAM – DISTANCE METRIC

- HISTCMP_CORREL: Measures the correlation between two histograms. A value of 1 indicates a perfect match, while a value of -1 indicates a complete mismatch.

$$d(H_1, H_2) = \frac{\sum_i (H_1(i) - \overline{H_1})(H_2(i) - \overline{H_2})}{\sqrt{\sum_i (H_1(i) - \overline{H_1})^2 \sum_i (H_2(i) - \overline{H_2})^2}}$$

- HISTCMP_CHISQR measure of the difference between the expected and observed frequencies of a set of categorical data. It is commonly used to quantify the dissimilarity between two histograms, a smaller value indicates a closer match.

$$d(H_1, H_2) = \sum_i \frac{\left(H_1(i) - H_2(i)\right)^2}{H_1(i)}$$

MULTIDIMENSION HISTOGRAM – DISTANCE METRIC

- HISTCMP_INTERSECT: Computes the intersection (minimum) between two histograms. A larger value indicates a closer match.

$$d(H_1, H_2) = \sum_i min\big(H_1(i), H_2(i)\big)$$

- HISTCMP_BHATTACHARYYA: is often used to quantify the dissimilarity between two probability distributions, such as histograms. Values closer to 0 indicate a closer match.

$$d(H_1, H_2) = \sqrt{1 - \frac{1}{\sqrt{\overline{H_1}\overline{H_2}B^2}\sum_i \sqrt{H_1(i).H_2(i)}}}$$

- HISTCMP_WEMD is used to quantify the dissimilarity between two histograms, which measures the minimum cost of transforming one distribution into another.

HISTOGRAM COMPARISION-THE SIMPLEST CLASSIFIER

1. Calculate the histograms of the two input images using cv2.calcHist()
2. Normalize the histograms computed above for the two input images using cv2.normalize()
3. Compare these normalized histograms using cv2.compareHist(). It returns the comparison metric value. Pass suitable histogram comparison method as parameter to this method.
4. For the Correlation and Intersection methods, the higher the metric, the more accurate the match. While for chi-square and Bhattacharyya, the lower metric value represents a more accurate match.

HISTOGRAM COMPARISION-THE VERY SIMPLE CLASSIFIER

```
def calcHistogram(frame, colorSpace = 'HSV',
        channels = [0, 1, 2], bins = [16, 16, 16], range=[0, 256, 0, 256, 0, 256]):
    if (colorSpace == 'HSV'):
        frame = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    elif (colorSpace == 'GRAY'):
        model_frame = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    elif (colorSpace == 'RGB'):
        frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    hist = cv2.calcHist([frame], channels, None, bins, range)
    hist = cv2.normalize(hist, hist).flatten()

    return hist
def histogramComparision(h1, h2, method='intersection'):
    if (method == "intersection"):
        comparison = cv2.compareHist(h1, h2, cv2.HISTCMP_INTERSECT)
    elif (method == "correlation"):
        comparison = cv2.compareHist(h1, h2, cv2.HISTCMP_CORREL)
    elif (method == "chisqr"):
        comparison = cv2.compareHist(h1, h2, cv2.HISTCMP_CHISQR)
    elif (method == "bhattacharyya"):
        comparison = cv2.compareHist(h1, h2, cv2.HISTCMP_BHATTACHARYYA)
    else:
        raise ValueError(
            'The method specified ' + str(method) + ' is not supported.')
    return comparison
```

ADVANTAGES OF COLOR HISTOGRAMS

- Fast and easy to compute
- Invariant to Translation, Rotation about the viewing axis
- Changes slowly with
  - Rotation about other axes (viewpoint changes)
  - Distance to object (scaling)
- Can be used for image matching in content-based image retrieval, object identification, and object location
- NOT invariant to changes in lighting conditions

COLORED DERIVATIVES

- Colored derivatives refer to the partial derivatives of an image with respect to its color channels. These derivatives are used to analyze the rate of change of pixel intensities along different directions in the image. It is often used to enhance the edges of an image or a frame in video.
- Colored derivatives are often used in tasks like edge detection, texture analysis, and feature extraction in computer vision applications.
- Local gradient magnitude indicates edge strength.

$$\left|Grad\left(f(x,y)\right)\right| = \sqrt{\left(\frac{\partial f(x,y)}{\partial x}\right)^2 + \left(\frac{\partial f(x,y)}{\partial y}\right)^2} \approx \left|\frac{\partial f(x,y)}{\partial x}\right| + \left|\frac{\partial f(x,y)}{\partial x}\right|$$

GRADIENT-BASED EDGE DETECTION NOTATIONS

- Gradient

$$Grad\big(f(x,y)\big) = \left[\frac{\partial f(x,y)}{\partial x}, \frac{\partial f(x,y)}{\partial y}\right]$$

- Gradient magnitude

$$\left|Grad\big(f(x,y)\big)\right| = \sqrt{\left(\frac{\partial f(x,y)}{\partial x}\right)^2 + \left(\frac{\partial f(x,y)}{\partial y}\right)^2} \approx \left|\frac{\partial f(x,y)}{\partial x}\right| + \left|\frac{\partial f(x,y)}{\partial x}\right|$$

- Gradient orientiation

$$\theta = tan^{-1}\left(\frac{\partial f(x,y)}{\partial y} \Big/ \frac{\partial f(x,y)}{\partial x}\right)$$

GRADIENT FILTERS

Operator	Matrix form
Central Diffrence	$\begin{bmatrix} 0 & 0 & 0 \\ -1 & [0] & 1 \\ 0 & 0 & 0 \end{bmatrix} \quad \begin{bmatrix} 0 & -1 & 0 \\ 0 & [0] & 0 \\ 0 & 1 & 0 \end{bmatrix}$
Roberts	$\begin{bmatrix} [0] & 1 \\ -1 & 0 \end{bmatrix} \quad \begin{bmatrix} [0] & 1 \\ -1 & 0 \end{bmatrix}$
Prewitt	$H_y = \begin{bmatrix} -1 & -1 & -1 \\ 0 & [0] & 0 \\ 1 & 1 & 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 1 & 1 \end{bmatrix} \quad H_x = \begin{bmatrix} -1 & 0 & 1 \\ -1 & [0] & 1 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \bullet \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$
Sobel	$H_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & [0] & 0 \\ 1 & 2 & 1 \end{bmatrix} = \begin{bmatrix} -1 \\ 0 \\ 1 \end{bmatrix} \bullet \begin{bmatrix} 1 & 2 & 1 \end{bmatrix} \quad H_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & [0] & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \bullet \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}$
Orientation Sobel	$H_y = \begin{bmatrix} 0 & 1 & 2 \\ -1 & [0] & 1 \\ -2 & -1 & 0 \end{bmatrix} \quad H_x = \begin{bmatrix} 0 & -1 & -2 \\ 1 & [0] & -1 \\ 2 & 1 & 0 \end{bmatrix}$

COLORED DERIVATIVES AS GLOBAL FEATURES

```
# Function to compute colored derivatives using Sobel operator
def compute_colored_derivatives(image):
    # Convert the image to the LAB color space
    lab_image = color.rgb2lab(image)
    # Compute derivatives for each LAB channel
    sobel_dx = np.zeros_like(lab_image)
    sobel_dy = np.zeros_like(lab_image)
    for i in range(3):  # Process each LAB channel
        sobel_dx[:, :, i] = sobel(lab_image[:, :, i], axis=0)
        sobel_dy[:, :, i] = sobel(lab_image[:, :, i], axis=1)
    return sobel_dx, sobel_dy
```

```
image = io.imread("image.jpg")
sobel_dx, sobel_dy = compute_colored_derivatives(image)
magnitude = np.sqrt(sobel_dx**2 + sobel_dy**2)
feature_vector = np.concatenate([sobel_dx.flatten(), sobel_dy.flatten()])
#Load images and  convert them by using feature_vector
svm_classifier = SVC()
svm_classifier.fit(X_train, y_train)
predictions = svm_classifier.predict(X_test)
```

HISTOGRAM BACK PROJECTION

- It is used for image segmentation or **finding objects** of interest in an image. It is used with object tracking algorithms like **meanshift** and **camshift**.
- It was proposed by Michael J. Swain, Dana H. Ballard in their paper *Indexing via color histograms*, Third international conference on computer vision,1990. This was one of the first works that use color to address the classic problem of Classification and Localisation in computer vision.
- Histogram Backprojection answers the question "*Where are the colors in the image that belong to the object being looked for (the target)?*"
- Histogram Backprojection involves the use of histograms and a backward projection process

HISTOGRAM BACK PROJECTION

- **Histogram** is used to represent the distribution of pixel intensities in an image.
- **Back Projection** involves projecting this histogram information back onto the image to highlight regions that match a given histogram.
- The name “Histogram back projection" essentially describes a process where a histogram is used to identify and highlight regions in an image that have a similar distribution to a specified reference histogram.
- This technique is often employed in object detection and tracking, where the goal is to find regions in an image that resemble a certain target based on their pixel intensity distribution.

HISTOGRAM BACK PROJECTION-ALGORITHM

1. Convert images (Object/ROI and target/overall image) into HSV and calculate their histograms

   roiHist = histogram of ROI; imgHist = histogram of an image.

```
roi = cv2.imread('..datasets/roi.jpg')
roi = cv2.cvtColor(roi,cv2.COLOR_BGR2HSV)
target = cv2.imread('..datasets/target.jpg')
target = cv2.cvtColor(target,cv2.COLOR_BGR2HSV)
roiHist = cv2.calcHist([roi],[0, 1], None, [180, 256], [0, 180, 0, 256] )
imgHist = cv2.calcHist([img],[0, 1], None, [180, 256], [0, 180, 0, 256] )
```

2. Find the ratio R = *roiHist*/*imgHist.* The ratio R is computed to compare how much a color in the ROI dominates compared to the overall image. R is size of $histNumBinsxhistNumBins$.

```
R = roiHist/(imgHist+epsilon)
```

   If a color is more frequent in the ROI than in the overall image, R is high for that color and vice versa.

HISTOGRAM BACK PROJECTION-ALGORITHM

3. Backproject R: Each pixel in the image is replaced by its corresponding probability R(h), forming a back project map that highlights regions likely to match the ROI. B(x,y) = R[h(x,y),s(x,y)] where h is hue and s is saturation of the pixel at (x,y). After that apply the condition B(x,y) = min[B(x,y), 1]. B(x, y) forms a back projection map, where each pixel's intensity represents the probability of being part of the target. The back project map assigns probabilities to each pixel based on how likely it is to belong to the ROI's histogram.

```
h, s, v = cv2.split(img)
B = R[h.flatten(),s.flatten()]
B = np.minimum(B,1)
B = B.reshape(target.shape[:2])
```

For each pixel at position at (i, j), the corresponding hue and saturation values are used to look up the ratio R and assign it to the corresponding pixel in the B image.

HISTOGRAM BACK PROJECTION-ALGORITHM

4. A convolution kernel is applied to smooth out noise in the back projection mapFine-tuning B by applying a convolution with a circular disc, B = D * B, where D is the disc kernel.

```
disc = cv2.getStructuringElement(cv2.MORPH_ELLIPSE,(5,5))
cv2.filter2D(B, -1, disc, B)
B = np.uint8(B)
B = cv2.normalize(B, B, 0, 255, cv2.NORM_MINMAX)
```

5. Now the location of maximum intensity gives us the location of object. If we are expecting a region in the image, thresholding for a suitable value gives a nice result.

```
ret,thresh = cv2.threshold(B, 10, 255, 0)
 # Overlay images using bitwise_and (bitwise_or, bitwise_xor)
thresh = cv2.merge((thresh, thresh, thresh))
res = cv2.bitwise_and(img, thresh)
```

HISTOGRAM BACK PROJECTION-EXAMPLE

$$Roi = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}; Img = \begin{bmatrix} 0 & 1 & 2 & 1 \\ 1 & 0 & 1 & 2 \\ 2 & 2 & 0 & 0 \\ 0 & 2 & 1 & 0 \end{bmatrix};$$

1. Compute Histograms: $histROI = [2/4 \quad 2/4 \quad 0]; histImg = [6/16 \quad 5/16 \quad 5/16];$
2. Compute the Ratio: $R(h) = [1.33 \quad 1.6 \quad 0]$
3. Back Project: $backProject =$

$$\begin{bmatrix} 1.33 & 1.6 & 0 & 1.6 \\ 1.6 & 1.33 & 1.6 & 0 \\ 0 & 0 & 1.33 & 1.33 \\ 1.33 & 0 & 1.6 & 1.33 \end{bmatrix}$$

4. Convolution (loccalsum) with $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$

$$backConv \approx \begin{bmatrix} 5.8 & 4.5 & 3.2 & 1.6 \\ 2.9 & 4.2 & 4.2 & 1.3 \\ 1.3 & 2.9 & 5.5 & 2.6 \\ 1.3 & 1.6 & 2.9 & 1.3 \end{bmatrix}$$

5. Find Maximum Intensity (or thresholding 0.35 to find the locations of ROI): The highest value in the backConv is 5.8 at (0, 0). The solution is

$$extractedRegion = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$$

HBP measures how well the histogram (distribution of pixel intensities) in a region of the target image matches the histogram of the ROI

HISTOGRAM BACK PROJECTION-STEP 3 WITH EXAMPLE

- Suppose we have an image of a scene with various objects, and we want to detect a red apple using histogram back projection.ROI contains 100 pixels, 50 pixels have a red intensity of 150, 30 pixels have a red intensity of 160, and 20 pixels have a red intensity of 170.
- For each pixel in the image, we calculate its likelihood of belonging to the apple based on its color similarity to the histogram. We do this by looking at the red intensity of each pixel and comparing it to the red intensity values in the histogram of the apple.
- For example, if a pixel in the image has a red intensity of 160, we check how many pixels in the apple's histogram have a similar intensity.
- Let's say in our histogram, there were 30 pixels with a red intensity of 160. We assign a higher probability to this pixel belonging to the apple.
- We repeat this process for every pixel in the image, calculating the probability of each pixel belonging to the apple based on its color similarity to the apple's histogram

HISTOGRAM BACK PROJECTION-STEP 3 WITH EXAMPLE

- Back project (*B = R[h.flatten(),s.flatten()]*): using the histogram of the apple, we back-project it onto the entire image. For each pixel in the image, we calculate its likelihood of belonging to the apple based on its color similarity to the histogram

```
probability = np.zeros_like(hsvt[:,:,0], dtype=np.float32)
    for i in range(img.shape[0]):
        for j in range(img.shape[1]):
            h = img[i, j, 0]
            s = img[i, j, 1]
            bin_h = int(h * 180 / 256)
            bin_s = int(s * 256 / 256)
            probability[i, j] = roiHist[int(bin_h, bin_s]
cv2.normalize(probability, probability, 0, 255, cv2.NORM_MINMAX).
```

HISTOGRAM BACK PROJECTION IN OPENCV

- OpenCV provides an inbuilt function:

cv2.calcBackProject(target_img, channels, roi_hist, ranges, scale)

```
# calculating object histogram
M = cv2.calcHist([hsvr],[0, 1], None, [180, 256], [0, 180, 0, 256] )

# normalize histogram and apply backprojection
cv2.normalize(M,M,0,255,cv2.NORM_MINMAX)
B = cv2.calcBackProject([hsvt],[0,1],M,[0,180,0,256],1)
```

The *cv2.calcBackProject* do some more operations such as: Edge Handling, Optional Scaling, ... to get better results than our manual implementation.